WEEK 9 – PROJECT PROGRESS REPORT

PRJ-18 | REAL-TIME UNIVERSITY BUS TRACKING SYSTEM

Driver Approval, Notifications and Admin Routes & Stops Integration

Student Name	**Birendra Prasad**
Course / Programme	MCA – Cyber Security
Project	PRJ-18
Reporting Period	Week 9
Project Repository	https://github.com/BirendraPrasad/University-Bus-Tracking

1. Introduction

This report presents the work completed during Week 9 of PRJ-18, the Real-Time University Bus Tracking System. Development focused on the driver account approval workflow, account-event notifications, continued verification of GPS data processing, and Admin Dashboard route/stop creation. Existing Bus and Driver dashboard functionality was retained during these changes.

2. Week 9 Objectives

- Implement administrator-controlled driver registration approval.
- Generate and store notifications for important driver and user account events.
- Verify that an approved driver can log in and view notifications.
- Continue checking GPS message processing through Kafka and persistence in PostgreSQL.
- Connect Add Route and Add Stop forms in the Admin Dashboard to backend APIs.
- Record test evidence and identify remaining development tasks.

3. Work Completed

3.1 Driver Registration Request and Approval

A driver registration request workflow was implemented. A driver who submits a public registration request remains pending until an administrator reviews the request. Pending or rejected driver accounts are prevented from using the normal driver login flow.

- Created driver registration request storage in PostgreSQL.
- Added GET /users/driver-requests to retrieve requests for administrator review.
- Added PUT /users/driver-requests/{request_id}/approve and PUT /users/driver-requests/{request_id}/reject.
- Enabled driver login after the request is approved.

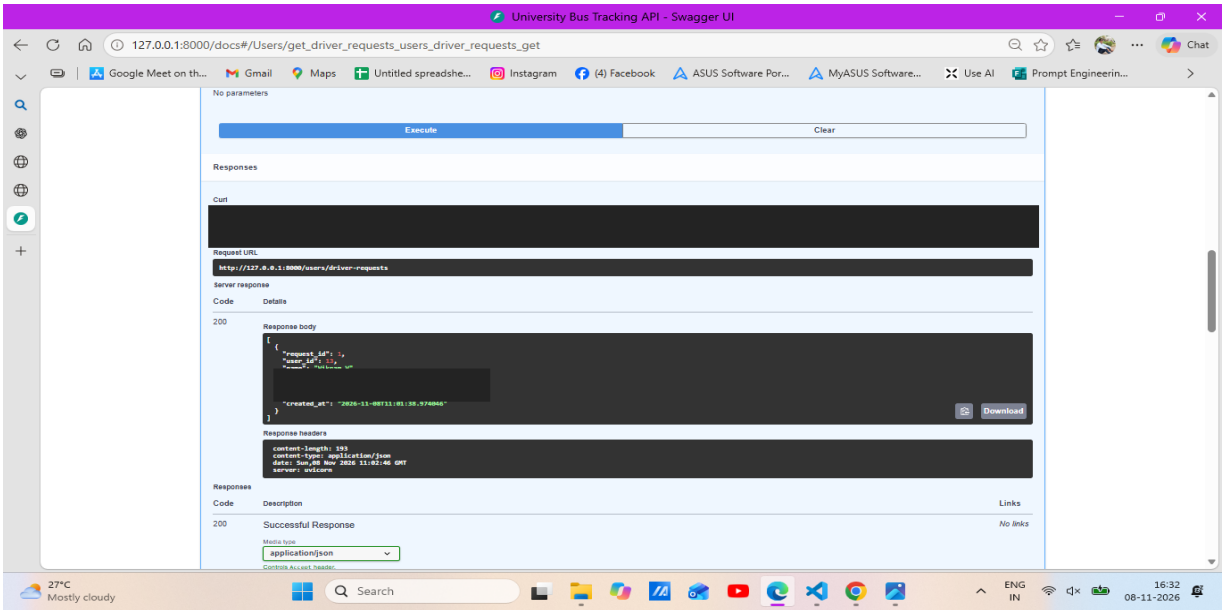


Figure 1. Admin API – Pending Driver Registration Request

Observed Output: The GET driver-requests API returned a pending request. The screenshot has been redacted to hide the bearer token and contact details.

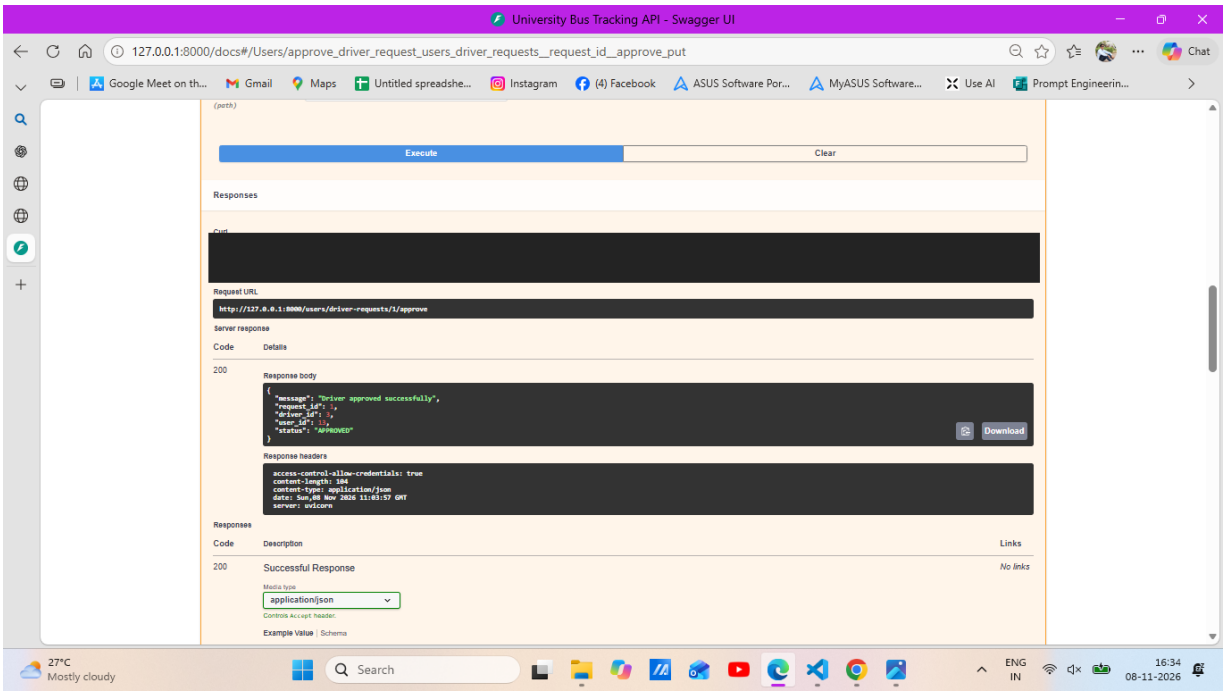


Figure 2. Admin API – Driver Approval Successful Response

Observed Output: The approval endpoint returned HTTP 200 with the message “Driver approved successfully” and status APPROVED. The bearer token has been redacted.

3.2 Approved Driver Login Verification

After approval, the driver account was tested through the login flow. The approved driver reached the Driver Dashboard. The screenshot confirms access to the dashboard; it does not indicate that a bus or trip was assigned.

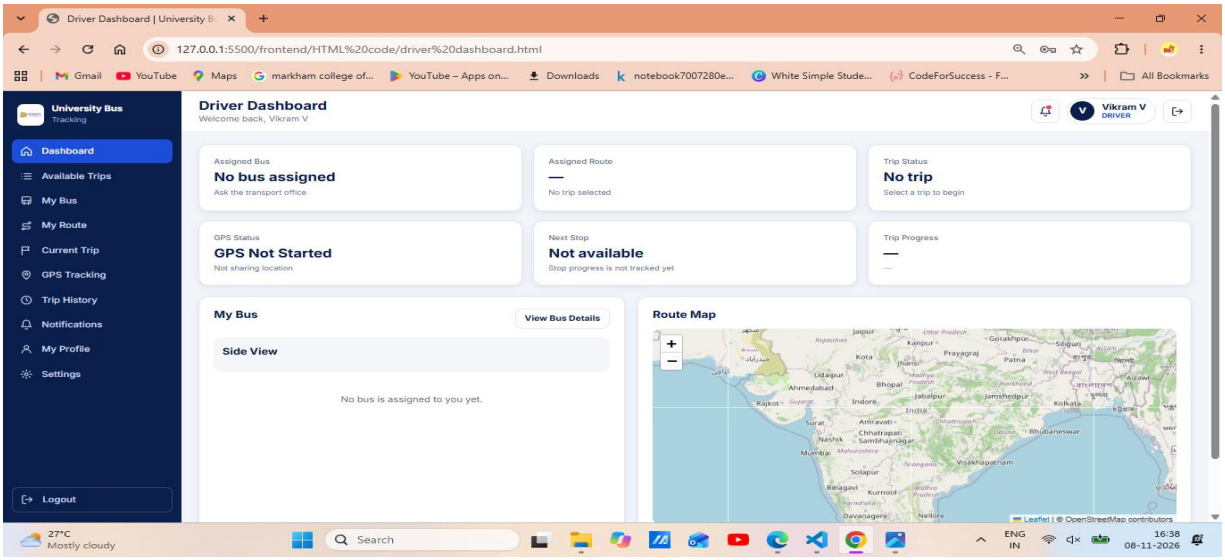


Figure 3. Driver Dashboard – Approved Driver Logged In

Observed Output: The dashboard displays the authenticated driver account (Vikram V). The page also shows that no bus and no trip are currently assigned, so those functions remain separate pending work.

3.3 Driver Notification Feature

Notifications are stored in the PostgreSQL notifications table and retrieved by the Driver Dashboard. The tested event types include New Driver Registration for administrators, Driver Registration Approved for the driver, and User Login for administrators. The notification records were also checked in PostgreSQL during testing; however, no database screenshot is included in this report.

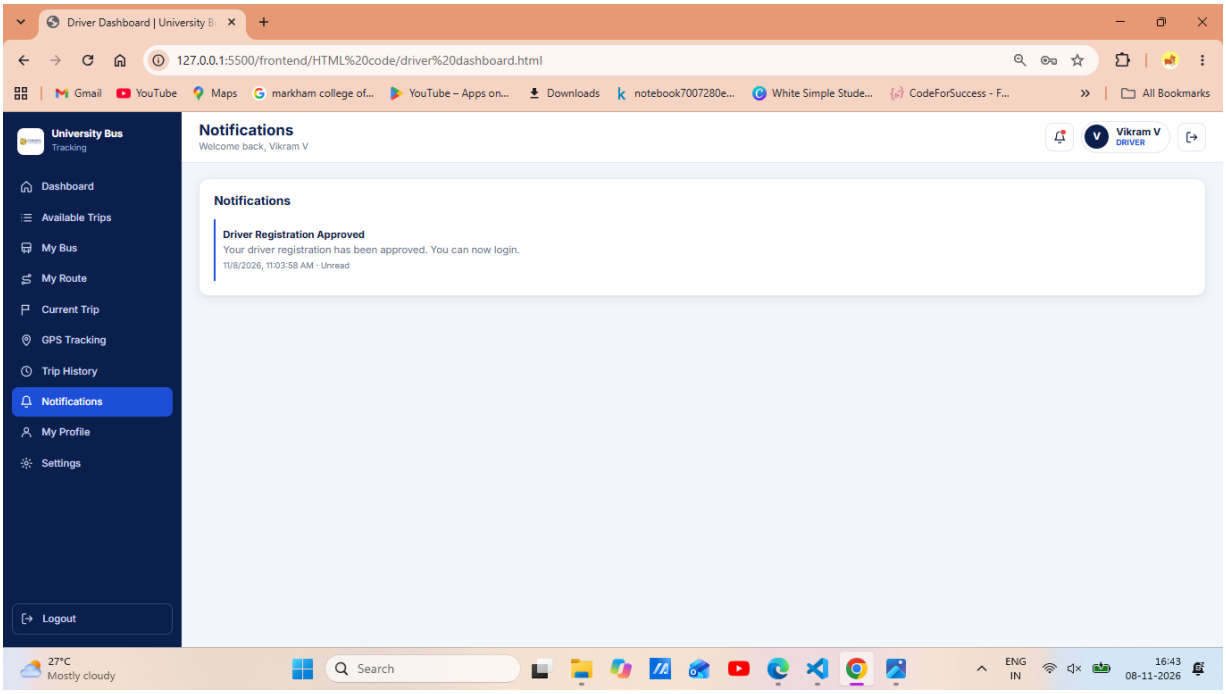


Figure 4. Driver Dashboard – Registration Approved Notification

Observed Output: The Driver Dashboard displays “Driver Registration Approved” with the approval message and an Unread status. The notification bell indicator is also visible.

3.4 GPS and Kafka Data Pipeline Verification

The GPS data pipeline was continued and checked. The tested data path is GPS Source → FastAPI → Kafka topic (bus-location) → GPS Consumer → PostgreSQL (bus_locations). Project testing notes recorded that Kafka messages were received and processed by the consumer and location records were saved in PostgreSQL, including data associated with Trip ID 6. No terminal or database screenshot is included in this report, so this section records the testing outcome without presenting screenshot evidence.

Important limitation: Kafka message reception and database persistence do not by themselves confirm continuous live marker movement on the Student Dashboard. Student-side live map behaviour and ETA must be tested separately.

3.5 Admin Dashboard – Routes & Stops Integration

The Routes & Stops section was extended to create route and stop records through the Admin Dashboard while retaining the existing route listing and stop viewing behaviour. The Add Route form was connected to POST /users/routes and the Add Stop form was connected to POST /users/stops.

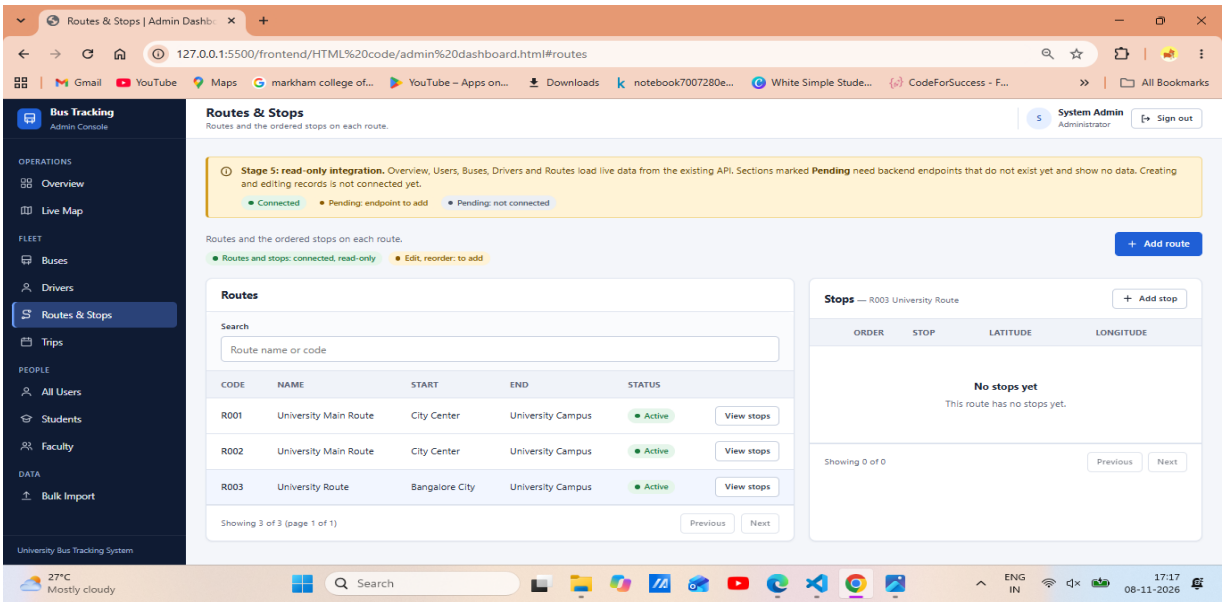


Figure 5. Admin Dashboard – R003 Route Listed

Observed Output: The Routes table displays R001, R002 and the newly created R003 – University Route. R003 is selected in the Stops panel and initially has no stops.

3.5.1 Add Stop – API Integration and Test

The first stop for route R003 was submitted through the Admin Dashboard. After saving, the Stops table displayed the saved stop with its order and coordinates.

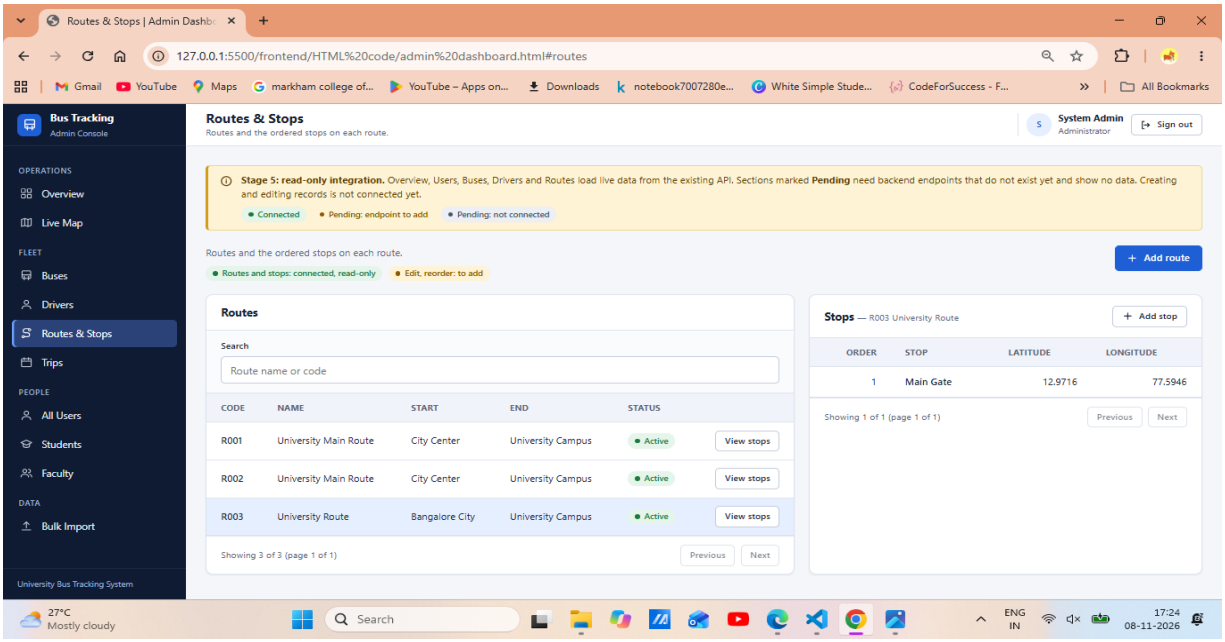


Figure 6. Admin Dashboard – Main Gate Stop Saved Under R003

Observed Output: The Stops table shows Main Gate under R003 with Stop Order 1, Latitude 12.9716 and Longitude 77.5946. This confirms the tested first-stop creation; multiple-stop sequencing still requires further testing.

Multiple stops can be associated with a route by assigning different stop-order values. The first stop has been tested for R003. Adding and validating further stops, automatic next stop order and duplicate-order handling remain future tasks.

4. Testing and Verification Summary

Test ID	Test Activity	Observed Result	Status
W9-01	Retrieve driver registration request	Pending request retrieved through Admin API.	PASS
W9-02	Approve driver request	Approval API returned HTTP 200 and APPROVED status.	PASS
W9-03	Approved driver login	Approved driver reached Driver Dashboard.	PASS
W9-04	Driver approval notification	Approval notification displayed with Unread status.	PASS
W9-05	Notification database check	Notification records observed in PostgreSQL.	VERIFIED
W9-06	Kafka GPS consumer	GPS messages received and processed.	VERIFIED
W9-07	GPS database persistence	Location records observed in PostgreSQL.	VERIFIED
W9-08	Add Route	R003 saved and displayed in Routes table.	PASS
W9-09	Add Stop	Main Gate saved and displayed under R003.	PASS

5. Technologies and Tools Used

Technology / Tool	Purpose
Python	Backend logic and GPS consumer implementation
FastAPI	REST API development and backend integration

PostgreSQL	Users, driver requests, notifications, routes, stops and GPS data storage
Apache Kafka	GPS location message streaming
JavaScript	Admin Dashboard form submission and API integration
HTML and CSS	Dashboard user interface
Ubuntu WSL2	Development and testing environment
Git and GitHub	Version control and source code repository

6. Pending Tasks

- Add and test multiple stops under R003 and other routes.
- Implement automatic next Stop Order and duplicate-order validation.
- Connect and test the Create Trip form in the Admin Dashboard.
- Implement the Admin Notifications interface.
- Verify continuous GPS marker updates on the Student Dashboard.
- Implement and validate ETA calculation using current GPS location, route and stop information.
- Complete Apache Superset integration for project analytics.
- Perform security testing, API testing, integration testing and end-to-end regression testing.
- Complete deployment and final project documentation.

7. Outcome of Week 9

Week 9 produced working improvements in driver account administration, notification handling and Admin Dashboard route/stop management. A pending driver request was retrieved and approved through the administrator workflow, the approved driver reached the Driver Dashboard, and the approval notification was displayed. The GPS message pipeline continued to be checked through Kafka and PostgreSQL. In the Admin Dashboard, route R003 and its first stop, Main Gate, were successfully created and displayed. Existing Bus and Driver dashboard functionality was preserved.

8. Plan for Week 10

- Add more stops to routes and test the stop sequence.
- Implement automatic Stop Order handling.
- Connect and test Create Trip from the Admin Dashboard.
- Continue Student Dashboard live GPS map integration.
- Start or continue ETA calculation after validating route and stop data.
- Proceed with the Admin Notifications interface and remaining dashboard integrations.

9. Conclusion

During Week 9, the Real-Time University Bus Tracking System progressed through driver registration approval, notification events, GPS message processing and Admin Dashboard route/stop creation. The successful creation of route R003 and its first stop demonstrates that route and stop records can be created from the dashboard and displayed from backend data. The next phase will focus on multiple-stop handling, trip creation, Student Dashboard live tracking, ETA, analytics and broader system testing.

Evidence and privacy note: Six original screenshots from the project work have been embedded. API bearer-token areas and driver contact details visible in API screenshots have been covered before insertion. No screenshot has been fabricated for the GPS pipeline or PostgreSQL notification query; those results are described as previously observed tests only.